

AuditTrace

Multi-Agent LLM Consensus for Infrastructure-as-Code Compliance Auditing

CMPE 258 Deep Learning | Final Project | San Jose State University | Spring 2026

Field	Value
Team ID	Group 4
Project Title	AuditTrace — Multi-Agent LLM Consensus for IaC Compliance Auditing
Project Track	Application
Focused Areas	Multi-agent LLM systems · LLM-as-judge evaluation · Compliance/security tooling · Hybrid symbolic + neural architectures
GitHub Repository	https://github.com/ParthGala2k/AuditTrace
Demo Video	https://youtu.be/3yNg2zou0WI

Team Members

Name	SJSU ID	Email	Role
Parth Gala	018198661	parth.gala@sjsu.edu	Sole contributor full-stack implementation, evaluation, documentation

Abstract

AuditTrace is a hybrid compliance auditing system that combines a deterministic infrastructure-as-code scanner (Checkov) with a three-model Large Language Model consensus filter that triages false positives using the surrounding Terraform context. The system addresses a concrete pain point in cloud security: static scanners produce a firehose of alerts 385 findings on the deliberately vulnerable terragoat repository alone without distinguishing genuine violations from contextual false positives. We send each Checkov finding plus its enclosing Terraform resource block to GPT-4o mini, DeepSeek V3, and Llama 3.1 70B in parallel; each model independently labels the finding as GENUINE, FALSE_POSITIVE, or UNCERTAIN, and a consensus vote assigns a confidence tier. On a hand-labeled evaluation set of 30 adversarial findings spanning 13 distinct Checkov rules, the consensus reaches 100% overall accuracy with 100% false-positive reduction and 100% true-positive retention, while individual models score F1 between 84.6% (Llama) and 100% (DeepSeek). Cohen's kappa between model pairs ranges 0.67 to 0.93, demonstrating that the models genuinely disagree on hard cases and that consensus voting performs measurable error correction rather than rubber-stamping. A separate planner benchmark (n=55 CIS clauses) quantifies an explicit accuracy-vs-latency-vs-cost tradeoff across the three models. All numbers are reproducible from two committed scripts and a SHA-256 prompt cache.

1. Introduction and Problem Description

Cloud compliance auditing is increasingly automated through static analysis of infrastructure-as-code (IaC) artifacts such as Terraform, CloudFormation, and Kubernetes manifests. Mature scanners Checkov, tfsec, kics encode hundreds of rules drawn from frameworks like the CIS Benchmarks, NIST SP 800-53, SOC 2, and PCI-DSS. These scanners are extremely good at one thing: rule-based pattern matching against the abstract syntax of an IaC file. They flag every occurrence of a violation deterministically. They are also, by design, completely context-blind.

On real engineering teams this produces a recurring failure mode: scanners surface hundreds of alerts per pull request, the vast majority of which are not actually actionable in the specific context they appear in. A bucket marked public-read in a marketing static-site module is fundamentally different from one in a production billing module, yet both look identical to the rule engine. Engineers stop looking at the alerts. The compliance posture of the codebase silently degrades.

We hypothesize that this is the precise problem class where current large language models add real value. LLMs cannot replace the rule engine they would hallucinate, miss obvious violations, and produce nondeterministic output but they are unusually well-suited to reading surrounding code and judging whether a flagged pattern actually represents a real risk in its context. We call this the false-positive filter task. It is the inverse of detection: detection is symbolic and deterministic, filtering is contextual and probabilistic.

Target users and scenario

- Cloud security engineers who triage Checkov output today and want a high-confidence shortlist of genuine violations.

- Compliance auditors who need a traceable mapping from policy clause (CIS, NIST, SOC 2) to specific lines of code in a customer repository.
- DevOps teams running IaC pipelines who want a CI gate that blocks pull requests introducing new high-confidence violations without drowning developers in noise.

Why this is a deep learning problem

The judgement task does this flagged Terraform block represent a real violation, given the file path, the comments, the sibling resources, the naming conventions? is fundamentally a contextual reasoning problem that classical NLP or rule systems cannot do. It requires the kind of broad domain understanding present in modern instruction-tuned LLMs. By framing the LLM as a judge rather than a detector, we avoid the failure modes (hallucination, miss-rate, nondeterminism) that have stopped LLMs from being used in compliance pipelines, while preserving the strength (contextual reasoning) that classical methods lack. We then combine three independent LLM judges into a single consensus vote, hypothesizing that distinct training corpora produce non-overlapping error modes that can be cancelled by aggregation.

2. Background and Related Work

AuditTrace draws on three lines of prior work. First, the field of automated infrastructure-as-code scanning, exemplified by Checkov [1] and the underlying Bridgecrew rule corpus, provides the deterministic detection layer on which we build. These tools encode security best practices as regular-expression-style checks over HCL, JSON, and YAML; they are mature but explicitly context-blind. The CIS Foundations Benchmarks [2] supply the policy framework we distill into machine-readable form.

Second, the recent literature on LLM-as-judge methodology [5] shows that using an LLM to evaluate another model's output, or to validate the output of a rule-based system, can produce evaluation signals comparable to human raters at a fraction of the cost. We adapt this pattern by treating the Checkov rule engine as the system-under-evaluation and the LLMs as the judges. Multi-model agreement quantified via Cohen's kappa [3], a classical inter-rater reliability statistic, gives us a calibrated measure of how much the LLMs disagree on hard cases, and the Landis & Koch [4] interpretation scale provides a published threshold (substantial vs almost-perfect agreement) for reporting it.

Third, agent-orchestration frameworks LangGraph [6] for stateful multi-step pipelines and OpenRouter [7] as a unified API gateway across LLM providers provide the engineering substrate that lets us run three models from different vendors against the same prompt with a single key. Our use is deliberately conservative: a two-node StateGraph (scan → consensus) rather than a deeply recursive agent loop, with all model responses cached on a SHA-256 hash of inputs for determinism.

Where this work differs from existing literature is in the framing. Most LLM-for-security papers position the LLM as a detector that competes with rule-based scanners. Our experiments demonstrate that this framing is mathematically dangerous: in a clause-mapping formulation (which we originally implemented) the precision of any LLM-generated finding is forced to 100% by construction, because the LLM can never invent `check_ids` that Checkov didn't already produce. The headline accuracy numbers in that setup are rigged. Reframing the LLM as a triage filter on top of the scanner never adding findings, only labelling and suppressing them yields metrics that genuinely vary across models and reflect real reasoning capability.

3. System, Model, and Algorithm Design

Overall architecture

The system has two phases. An offline phase, run once, distills the 312-page CIS AWS Foundations Benchmark PDF into 89 structured compliance requirements using GPT-4o mini (one LLM call per detected control heading), and saves the result to `policies/cis_aws_v7.json`. This is committed to the repository and never regenerated.

A runtime phase, run per audit, is implemented as a two-node LangGraph StateGraph. The scan node clones the target GitHub repository, runs Checkov over it, extracts the full enclosing Terraform resource block for each finding (via a brace-balanced back-walk from the violation line), attaches the HCL block to the finding payload, and deletes the local clone. The consensus node passes the enriched findings to three LLMs running in parallel through OpenRouter GPT-4o mini, DeepSeek V3, and Llama 3.1 70B Instruct. Each model independently labels every finding as `GENUINE`, `FALSE_POSITIVE`, or `UNCERTAIN` and supplies a one-line reason. The verdicts are aggregated into a tier:

GENUINE votes	Tier	Action
3 / 3 GENUINE	HIGH	Prioritize fix immediately
2 / 3 GENUINE	LIKELY	Schedule fix
1 / 3 GENUINE	UNCERTAIN	Manual review
2 / 3 FALSE_POSITIVE	LIKELY_FP	Probably noise, but keep visible
3 / 3 FALSE_POSITIVE	SUPPRESSED	Auto-removed from violations list

Key technique: HCL context extraction

Checkov returns the line range of a violation but not the enclosing resource. Our `extract_resource_block()` function in `tools/checkov_runner.py` walks backwards from the violation line until it finds a line beginning with `resource`, `module`, `data`, `provider`, or `variable`, then walks forward through balanced braces until the enclosing block closes. This produces a self-contained Terraform snippet that the LLM can read without needing the rest of the file. For non-HCL inputs (CloudFormation JSON, Kubernetes YAML) the implementation falls back to a windowed snippet of ± 5 lines, degrading gracefully.

Key technique: SHA-256 prompt cache

LLM responses are cached on disk at `backend/mapping_cache/`. The cache key is the SHA-256 hash of `(TASK_VERSION, model, sorted check_ids, sorted clause_ids, HCL digest)`. `TASK_VERSION` is a string constant in `agents/consensus.py` that we bump whenever the prompt or output schema changes this invalidates older cache entries automatically. The result is that every reported number is fully deterministic on re-runs, even though the underlying LLMs are sampled with temperature parameters outside our control on the server side.

Key technique: parallel multi-model voting

All three model judgements for a given audit are dispatched in parallel via `asyncio.gather()`; the wall-clock latency is approximately the slowest model's latency (typically DeepSeek at 7 seconds per batch of 25 findings) rather than the sum. The models cannot communicate with each other during the judgement step

there is no cross-talk, no chain-of-thought sharing, no debate protocol. This deliberate independence is what makes the consensus signal meaningful: if all three converge on the same verdict without coordination, the signal is strong; if they disagree, the disagreement is genuine and carries information.

4. Implementation Details

Languages, frameworks, and libraries

Layer	Technology
Agent orchestration	LangGraph 0.2.x (two-node StateGraph)
LLM provider	OpenRouter (OpenAI-compatible REST API)
laC scanner	Checkov 3.x via subprocess
PDF parsing	PyMuPDF (offline policy distillation only)
Backend framework	FastAPI + Uvicorn
Backend language	Python 3.13
Persistence	SQLite (audits + versions)
Frontend framework	React 18 + Vite
Frontend language	JavaScript (JSX)
Inter-rater statistic	Cohen's kappa (custom implementation)
Caching	SHA-256 keyed JSON cache on disk

System setup

Development was carried out on a Windows 11 workstation with Python 3.13 in a virtual environment under `backend/venv/`. No GPU is required; all model inference runs in the OpenRouter cloud. Local CPU is used only for cloning the target repository, running Checkov as a subprocess (~26 seconds for the terragoat repo, ~6 seconds for smaller test repos), and computing the consensus aggregation. The frontend is served by Vite on port 5173 with a proxy that forwards `/audit`, `/eval`, `/policies`, `/history`, and `/fix` requests to FastAPI on port 8000.

Important implementation decisions

- Temperature set to 0 for all LLM calls. Combined with the SHA-256 prompt cache, this gives bit-identical reproducibility on re-runs.
- Findings are batched 25 per LLM call. Smaller batches than the original 80 because the new prompt includes the full HCL block per finding, which expands token consumption.
- All three models judge every finding (no early-exit). Single-model dropout would lose the consensus signal we are trying to measure.
- Annotation file `fp_annotations.json` is hand-crafted with adversarial pairs two entries sharing the same `check_id` but differing in surrounding context, one labelled `GENUINE` and one `FALSE_POSITIVE`. This forces the LLMs to actually use the context rather than memorising rule outcomes.
- The `/evaluate` endpoint from the original architecture is retained for backwards compatibility but the new `/eval/fp-filter` and `/eval/planner` endpoints serve precomputed JSONs to the frontend EvalPanel.
- No credentials are committed; the `.env` file is gitignored and the GitHub push-protection scan was used to verify no live secrets appear in the annotation set.

Code location

Full source code is available at:

<https://github.com/ParthGala2k/AuditTrace>

5. Task Distribution and Contributions

This was a single-author project (Group 4 with one student). All components were designed, implemented, evaluated, and documented by Parth Gala.

Component	Contributor
Backend pipeline (main.py, consensus.py, checkov_runner.py, db.py)	Parth Gala
Frontend application (App.jsx, IssueCard, EvalPanel, design system)	Parth Gala
Evaluation suite (eval_e2e.py, run_eval.py, compute_metrics.py)	Parth Gala
Hand-labeled FP annotation set (30 entries across 13 Checkov rules)	Parth Gala
CIS AWS policy distillation (89 structured requirements)	Parth Gala
Demo deck (make_ppt.py, AuditTrace_Presentation.pptx)	Parth Gala
Project report (this document) and README	Parth Gala
Video demo recording	Parth Gala

6. Evaluation and Testing Results

Methodology

We evaluated AuditTrace on two independent benchmark suites with hand-labeled ground truth. Both produce deterministic JSON outputs that are committed to the repository and regenerable with a single command, allowing the report numbers to be independently verified by anyone with an OpenRouter key.

Suite 1 — FP-Filter Evaluation

Thirty hand-curated findings (15 GENUINE + 15 FALSE_POSITIVE) spanning 13 distinct Checkov check IDs across S3, IAM, RDS, EBS, VPC, ELB, and CloudTrail rules. Each annotation pair is adversarial: two entries that share the same check_id but differ in surrounding HCL context, one correctly genuine and one a contextual false positive. We compute per-model precision, recall, and F1 on the GENUINE class; the same metrics on the FALSE_POSITIVE class; Cohen's kappa between every pair of models on the 3-way verdict label; per-tier precision (does the HIGH tier actually mean genuine?); FP reduction rate; TP retention rate; and overall consensus accuracy.

Per-model classification results (GENUINE class)

Model	Precision	Recall	F1	TP	FP	FN	TN
GPT-4o mini	93.8%	100.0%	96.8%	15	1	0	14
DeepSeek V3	100.0%	100.0%	100.0%	15	0	0	15
Llama 3.1 70B	100.0%	73.3%	84.6%	11	0	4	15

Per-model F1 ranges from 84.6% (Llama 3.1 70B) to 100% (DeepSeek V3). Llama exhibits an under-flagging failure mode high precision but low recall, missing four real violations on this set. GPT-4o mini exhibits a single over-flagging error. DeepSeek achieves zero errors on n=30. The substantive finding is the variance: if we deployed any single model in isolation, we would be wrong somewhere between zero and four times on this evaluation set. Consensus voting recovers all of these individual errors.

Per-model classification results (FALSE_POSITIVE class)

Model	Precision	Recall	F1
GPT-4o mini	100.0%	93.3%	96.6%
DeepSeek V3	100.0%	100.0%	100.0%
Llama 3.1 70B	78.9%	100.0%	88.2%

Inter-model agreement (Cohen's kappa)

Model pair	κ	Landis & Koch interpretation
gpt-4o-mini vs deepseek-chat	0.93	Almost perfect
gpt-4o-mini vs llama-3.1-70b-instruct	0.67	Substantial
deepseek-chat vs llama-3.1-70b-instruct	0.73	Substantial

Cohen's kappa ranges from 0.67 (GPT-4o mini vs Llama) to 0.93 (GPT-4o mini vs DeepSeek). No pair exhibits perfect agreement. This is the critical finding for justifying the use of three models: the models genuinely disagree on hard cases, which means the consensus mechanism is performing real error correction rather than rubber-stamping identical verdicts.

Per-tier ground-truth distribution

Tier	Count	Truly Genuine	Truly FP	% Genuine
HIGH	11	11	0	100.0%
LIKELY	4	4	0	100.0%
LIKELY_FP	1	0	1	0.0%
SUPPRESSED	14	0	14	0.0%

Tier labels are predictive of ground truth on this set. Every finding assigned to the HIGH tier (11 findings) is in fact a genuine violation; every finding assigned to SUPPRESSED (14 findings) is in fact a false positive. The mapping from tier name to ground-truth class is therefore non-arbitrary the tier carries actual information about the underlying truth.

Consensus headline numbers

Metric	Value	Interpretation
FP reduction rate	100.0%	Fraction of true FPs correctly suppressed by consensus
TP retention rate	100.0%	Fraction of true GENUINES correctly retained
Overall accuracy	100.0%	Fraction of all 30 cases assigned to the correct tier

All three headline metrics are 100% on n=30. These three numbers are not independent they are three views of the same fact: every case is in the correct tier. The load-bearing evidence is the per-model variance above (84.6%-100% F1) and the Cohen's kappa range above (0.67-0.93). The headline 100% should be interpreted as a consequence of those underlying numbers on a small hand-curated set, not as evidence of an ideal system that will generalize unchanged to a much larger dataset.

Suite 2 — Planner Benchmark

A separate benchmark of 55 test cases drawn from the CIS AWS Foundations Benchmark PDF. Each test case provides a policy clause as input and the expected (severity, requirement_type, check_targets) triple as the ground truth. All three models are evaluated under identical prompting, with severity accuracy, requirement-type accuracy, targets F1, latency, total tokens, and dollar cost recorded.

Model	Severity Acc.	Type Acc.	Targets F1	Latency	Cost / run
GPT-4o mini	40.0%	85.5%	70.5%	1.34 s	\$0.0022
DeepSeek V3	38.2%	90.9%	80.9%	7.15 s	\$0.0018
Llama 3.1 70B	50.9%	90.9%	79.2%	3.04 s	\$0.0057

No single winner emerges. GPT-4o mini is the fastest at 1.34 seconds per call but has the lowest targets F1 (70.5%). DeepSeek V3 has the highest type accuracy (90.9%) and targets F1 (80.9%) at the lowest dollar cost (\$0.0018), but is the slowest at 7.15 seconds per call. Llama 3.1 70B sits in the middle on speed and is the only model to exceed 50% severity accuracy. Severity accuracy is intentionally low across all three models because severity is the most subjective field in the ground truth — reasonable auditors disagree on whether a given clause is critical or high. The objective signals (type accuracy and targets F1) show clearer differentiation.

How to reproduce these results

Every number in this section is reproducible from the committed repository. The full procedure is documented in README.md but the essential commands are:

1. `cd backend && python -m venv venv && .\venv\Scripts\Activate.ps1`
2. `pip install -r requirements.txt`
3. Create backend/.env with `OPENROUTER_API_KEY=<key>`
4. `python ../evaluation/eval_e2e.py` → writes `evaluation/eval_e2e_results.json`
5. `python ../evaluation/run_eval.py` → writes `evaluation/comparison_summary.json`

Cold-cache runtime is approximately 30–60 seconds for the FP-filter eval and 3–7 minutes for the planner benchmark. Warm-cache (after the first run) reruns complete in under one second because all LLM responses are SHA-256-keyed on disk.

Verification of correctness

- Determinism: re-running `eval_e2e.py` produces bit-identical `eval_e2e_results.json` under SHA-256 cache. Confirmed across multiple runs during development.

- End-to-end smoke test: a complete audit was run against bridgecrewio/terragoat producing 385 findings (211 high, 174 medium, 86.8% compliance score) with all three LLM tiers populated correctly. Screen recording of this run is in the demo video.
- Frontend smoke test: the /eval/fp-filter and /eval/planner endpoints were verified via the EvalPanel component in the React frontend. Manual inspection confirmed numbers match the committed JSONs.
- No-credential validation: GitHub push protection blocked an earlier commit containing a Stripe-pattern placeholder. The value was replaced with an obviously redacted placeholder and re-pushed cleanly.

Limitations of the evaluation

- $n = 30$ is small for the FP-filter set. The per-model F1 spread (84.6%-100%) and the Cohen's kappa range (0.67-0.93) are the most defensible findings; the 100% consensus accuracy is partly an artifact of dataset size.
- Annotations were created by a single annotator (the author). A multi-annotator agreement study would strengthen the ground truth.
- Severity accuracy in the planner benchmark is noisy because the ground-truth severity field is subjective. Type accuracy and targets F1 should be treated as the primary objective signals.
- The evaluation covers only Terraform/HCL inputs. CloudFormation and Kubernetes manifests are detected by Checkov but the HCL-extraction fallback degrades for them; their FP-filter quality has not been quantified.
- The system has been tested against the CIS AWS Foundations Benchmark only. Generalization to NIST SP 800-53, SOC 2, and PCI-DSS is future work.

7. References

- [1] Bridgecrew / Prisma Cloud. Checkov: open-source infrastructure-as-code static analysis. <https://www.checkov.io/>
- [2] Center for Internet Security. CIS Amazon Web Services Foundations Benchmark, v3.0. 2023. https://www.cisecurity.org/benchmark/amazon_web_services
- [3] Cohen, J. (1960). A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1), 37–46.
- [4] Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. Biometrics, 33(1), 159–174.
- [5] Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks.
- [6] LangChain Inc. LangGraph: stateful multi-agent LangChain framework. <https://github.com/langchain-ai/langgraph>
- [7] OpenRouter. Unified LLM API gateway. <https://openrouter.ai/docs>
- [8] Artifex Software. PyMuPDF: Python bindings for the MuPDF library. <https://pymupdf.readthedocs.io/>

- [9] Ramírez, S. FastAPI: modern high-performance Python web framework. <https://fastapi.tiangolo.com/>
- [10] Meta AI. Llama 3.1 model card. July 2024.
- [11] DeepSeek AI. DeepSeek-V3 technical report. 2024.
- [12] OpenAI. GPT-4o mini system card. July 2024.
- [13] bridgecrewio. terragoat: deliberately vulnerable Terraform reference repository.
<https://github.com/bridgecrewio/terragoat>
- [14] Vite. Frontend tooling for modern web projects. <https://vitejs.dev/>
- [15] Python Software Foundation. Python 3.13 language reference. <https://docs.python.org/3.13/>